

소프트웨어 설계서

AI 기반 개인 소비 분석 및 예산 관리 앱

GitHub: <https://github.com/lcanndoit/ai-budget-app>

항목	내용
프로젝트명	AI 기반 개인 소비 분석 및 예산 관리 앱
문서 번호	SDD-2026-001
버전	v1.0
작성일	2026년 5월 26일
작성자	손재연
소속	소프트웨어공학 과제
상태	최종본

목 차

1. 서론	3
1.1 목적 및 범위	3
1.2 용어 정의	3
1.3 참조 문서	4
2. 소프트웨어 아키텍처	4
2.1 정적 구조	4
2.2 동적 구조	5
3. 모듈/패키지 설계	6
3.1 Backend 패키지	6
3.2 AI 모듈	8
3.3 Frontend 패키지	9
4. 인터페이스 설계	10
4.1 외부 시스템 인터페이스	10
4.2 사용자 인터페이스	11
5. 데이터 설계	12
6. 구현 기술 설계	13
7. 요구사항 추적표	14

1. 서론

1.1 목적 및 범위

본 문서는 「AI 기반 개인 소비 분석 및 예산 관리 앱」(이하 'ai-budget-app')의 소프트웨어 컴포넌트에 대한 설계 사항을 기록하고 문서화하는 것을 목적으로 한다. 본 시스템은 사용자의 거래 내역을 수집하고, AI(Python 기반 머신러닝/규칙 모델)로 소비를 자동 분류하여 예산 초과 경보, 지출 패턴 분석, 카테고리별 예산 설정 기능을 제공한다.

본 문서의 범위는 Backend(Spring Boot), AI 모듈(Python), Frontend(React) 세 하위 시스템 전반을 포함하며, 각 컴포넌트의 정적·동적 구조, 인터페이스, 데이터 설계, 구현 기술을 기술한다.

1.2 용어 정의

용어	정의
거래 내역(Transaction)	사용자가 입력하거나 외부 연동을 통해 가져온 수입/지출 금액, 일시, 설명 정보
소비 분류(Category)	AI 모델이 거래 설명 텍스트를 분석하여 자동 부여하는 지출 카테고리 (식비, 교통 등)
예산(Budget)	사용자가 카테고리별로 설정한 월간 지출 한도 금액
API(Application Programming Interface)	시스템 컴포넌트 간 또는 외부 서비스와의 데이터 교환을 위한 규약
JWT(JSON Web Token)	사용자 인증 및 세션 관리에 사용하는 토큰 기반 인증 방식
REST(Representational State Transfer)	HTTP 프로토콜 기반의 무상태 아키텍처 스타일
SDD	Software Design Document, 소프트웨어 설계서

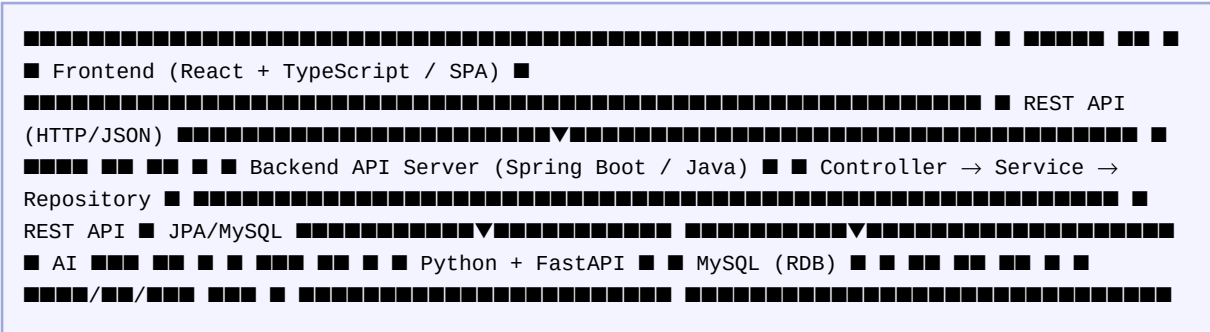
1.3 참조 문서

문서 번호	문서명
REQ-2026-001	요구사항 정의서 - ai-budget-app v1.0
SYS-2026-001	시스템 정의서 - ai-budget-app v1.0
GitHub README	https://github.com/lcanndoit/ai-budget-app/blob/main/README.md

2. 소프트웨어 아키텍처

2.1 정적 구조

본 시스템은 3-Tier 레이어드 아키텍처와 AI 마이크로서비스를 결합한 구조를 채택한다. Frontend(React SPA) → Backend API Server(Spring Boot) → AI 서비스(Python FastAPI) / DB(MySQL)로 계층이 분리되며, 각 계층은 REST API를 통해 통신한다.

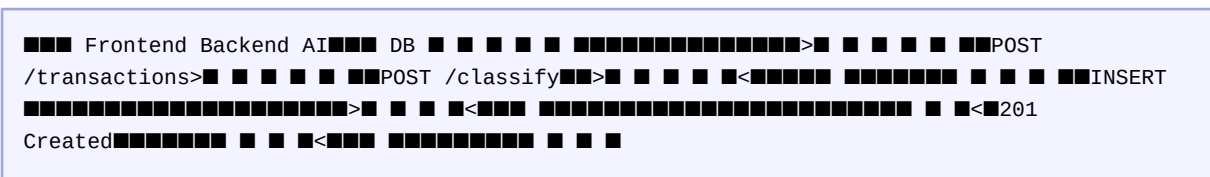


【컴포넌트 설명】

컴포넌트	역할	기술
Frontend	UI 렌더링, 사용자 입력 처리, 상태 관리	React, TypeScript
Backend API	비즈니스 로직, 인증, DB 접근, AI 연동	Spring Boot (Java)
AI 서비스	거래 텍스트 기반 소비 카테고리 자동 분류	Python, FastAPI
Database	사용자/거래/예산 데이터 영속 저장	MySQL

2.2 동적 구조

동적 구조는 주요 유스케이스인 「거래 등록 및 AI 분류」 시퀀스를 기준으로 기술한다.



① 사용자가 Frontend에서 거래 금액, 일시, 설명을 입력한다. ② Frontend는 Backend에 POST /api/transactions 요청을 전송한다. ③ Backend는 AI 서비스에 소비 분류를 요청하고, 결과 카테고리를 수신한다. ④ Backend는 분류된 카테고리들과 함께 거래를 DB에 저장한다. ⑤ Frontend에 201 Created 응답과 저장된 데이터를 반환한다.

3. 모듈/패키지 설계

3.1 SDD-P-001 : Backend (Spring Boot)

Backend는 MVC 패턴 기반으로 Controller / Service / Repository / Model 레이어로 구성된다.

패키지/클래스	역할
com.budget.controller	HTTP 요청 수신 및 응답 반환 (REST Controller)
TransactionController	거래 등록, 조회, 수정, 삭제 API 처리
BudgetController	예산 설정 및 조회 API 처리
UserController	회원가입, 로그인, 인증 API 처리
AnalysisController	소비 분석 결과 조회 API 처리
com.budget.service	비즈니스 로직 구현, 트랜잭션 관리
TransactionService	거래 CRUD, AI 분류 연동, 예산 초과 검사
BudgetService	카테고리별 예산 설정 및 집계
UserService	회원 관리, JWT 발급/검증
AnalysisService	월별/카테고리별 통계 계산
com.budget.repository	JPA 기반 DB 접근 인터페이스
TransactionRepository	거래 데이터 CRUD, 날짜/카테고리 조건 조회
BudgetRepository	예산 데이터 CRUD
UserRepository	사용자 정보 조회, 이메일 기반 검색
com.budget.model	엔티티 및 DTO 클래스
Transaction	거래 엔티티 (id, amount, description, category, date, userId)
Budget	예산 엔티티 (id, category, amount, month, userId)
User	사용자 엔티티 (id, email, password, name, createdAt)

3.1.1 주요 메서드 설계

메서드 ID	메서드명	입력	출력	설명
M-B-001	createTransaction()	TransactionDTO	TransactionDTO	AI 분류 포함 거래 생성
M-B-002	getTransactions()	userId, month	List<TransactionDTO>	월별 거래 목록 조회
M-B-003	setBudget()	BudgetDTO	BudgetDTO	카테고리별 예산 설정
M-B-004	getMonthlyAnalysis()	userId, month	AnalysisDTO	월별 소비 분석 반환
M-B-005	login()	email, password	JWT Token	사용자 인증 및 토큰 발급

3.2 SDD-P-002 : AI 모듈 (Python)

AI 모듈은 Python으로 구현된 독립 서비스로, FastAPI를 통해 REST 엔드포인트를 제공한다. 거래 설명 텍스트를 입력받아 소비 카테고리를 분류하는 기능을 수행한다.

모듈	역할
ai/main.py	FastAPI 앱 진입점, 라우팅 설정
ai/classifier.py	소비 분류 모델 로드 및 예측 로직
ai/preprocessor.py	텍스트 전처리 (형태소 분석, 정규화)
ai/models/	학습된 모델 파일 저장 디렉토리
ai/schemas.py	Pydantic 기반 입출력 스키마 정의
ai/categories.py	지원 카테고리 목록 및 매핑 정의

메서드 ID	메서드명	입력	출력	설명
M-A-001	classify()	description: str	CategoryResult	텍스트 기반 카테고리 분류
M-A-002	preprocess()	text: str	tokens: List[str]	텍스트 토큰나이징 및 정규화
M-A-003	load_model()	model_path: str	Model	분류 모델 파일 로드

【지원 소비 카테고리】

카테고리 코드	카테고리명	예시 거래 설명
CAT-01	식비	맥도날드, 편의점, 배달의민족
CAT-02	교통	지하철, 택시, 주유소, 카카오T
CAT-03	쇼핑	쿠팡, 무신사, 올리브영
CAT-04	문화/여가	영화, 넷플릭스, 게임 아이템
CAT-05	의료/건강	병원, 약국, 헬스장
CAT-06	통신	SKT, KT, LGU+ 요금
CAT-07	기타	분류 불가 또는 기타 지출

3.3 SDD-P-003 : Frontend (React)

Frontend는 React 기반 SPA로, 컴포넌트 계층 구조로 화면을 구성한다.

컴포넌트/모듈	역할
src/pages/Dashboard	메인 대시보드: 이번 달 지출 요약, 예산 달성률 차트
src/pages/Transactions	거래 내역 목록 조회, 거래 추가/수정 폼
src/pages/Budget	카테고리별 예산 설정 및 현황 시각화
src/pages/Analysis	월별/카테고리별 소비 분석 차트
src/pages/Login	로그인 / 회원가입 화면
src/components/Chart	소비 분석용 Bar/Pie 차트 공통 컴포넌트
src/components/TransactionForm	거래 입력 폼 컴포넌트
src/api/axiosClient	Axios 인스턴스, JWT 인터셉터 설정
src/store/	Redux/Context 기반 전역 상태 관리 (인증, 거래 데이터)

4. 인터페이스 설계

4.1 외부 시스템 인터페이스

인터페이스 ID	대상 시스템	방향	프로토콜	설명
IF-001	AI 분류 서비스	Backend → AI	HTTP/REST	POST /classify 로 텍스트 전달, 카테고리 수신
IF-002	MySQL DB	Backend ↔ DB	JDBC/JPA	거래, 예산, 사용자 데이터 CRUD
IF-003	Frontend	Frontend → Backend	HTTP/REST	JSON 기반 API 호출, JWT 인증 헤더 포함
IF-004	(선택) 외부 금융 API	Backend → 외부	HTTP/REST	카드사/은행 거래내역 자동 수집 (미래 확장)

【주요 REST API 엔드포인트】

HTTP 메서드	URI	요청 Body	응답	설명
POST	/api/auth/signup	email, password, name	UserDTO	회원가입
POST	/api/auth/login	email, password	JWT Token	로그인
GET	/api/transactions	-	List<Transaction>	거래 목록 조회
POST	/api/transactions	TransactionDTO	TransactionDTO	거래 등록 (AI 분류 포함)
PUT	/api/transactions/{id}	TransactionDTO	TransactionDTO	거래 수정
DELETE	/api/transactions/{id}	-	204 No Content	거래 삭제
GET	/api/budgets	-	List<Budget>	예산 목록 조회
POST	/api/budgets	BudgetDTO	BudgetDTO	예산 설정
GET	/api/analysis/monthly	month(query)	AnalysisDTO	월별 소비 분석

4.2 사용자 인터페이스

사용자 인터페이스는 React SPA 형태로 제공되며, 다음 5개의 주요 화면으로 구성된다.

화면 ID	화면명	주요 기능 및 UI 요소
UI-001	로그인/회원가입	이메일·비밀번호 입력 폼, 유효성 검사, JWT 토큰 저장
UI-002	대시보드	이번 달 총 지출 금액, 예산 달성률 게이지, 최근 거래 5건 목록
UI-003	거래 내역	거래 리스트(날짜/카테고리/금액), 거래 추가 모달, 필터(월/카테고리)
UI-004	예산 관리	카테고리별 예산 금액 입력 폼, 사용 현황 Progress Bar
UI-005	소비 분석	월별 지출 Bar Chart, 카테고리별 Pie Chart, 전월 대비 증감율

5. 데이터 설계

데이터는 MySQL 관계형 데이터베이스에 저장되며, Spring Data JPA를 통해 접근한다. 주요 엔티티는 User, Transaction, Budget이다.

DD-01 : users (사용자 테이블)

컬럼명	데이터 타입	PK/FK	NOT NULL	설명
id	BIGINT	PK (AI)	Y	사용자 고유 식별자
email	VARCHAR(100)	-	Y	사용자 이메일 (UNIQUE)
password	VARCHAR(255)	-	Y	bcrypt 암호화된 비밀번호
name	VARCHAR(50)	-	Y	사용자 이름
created_at	DATETIME	-	Y	계정 생성 일시

DD-02 : transactions (거래 내역 테이블)

컬럼명	데이터 타입	PK/FK	NOT NULL	설명
id	BIGINT	PK (AI)	Y	거래 고유 식별자
user_id	BIGINT	FK→users	Y	소유 사용자 ID
amount	DECIMAL(15,2)	-	Y	거래 금액 (양수: 수입, 음수: 지출)
description	VARCHAR(255)	-	Y	거래 설명 (AI 분류 입력값)
category	VARCHAR(50)	-	Y	AI 또는 수동 지정 카테고리
transaction_date	DATE	-	Y	거래 발생 날짜
created_at	DATETIME	-	Y	레코드 생성 일시

DD-03 : budgets (예산 테이블)

컬럼명	데이터 타입	PK/FK	NOT NULL	설명
id	BIGINT	PK (AI)	Y	예산 고유 식별자
user_id	BIGINT	FK→users	Y	소유 사용자 ID
category	VARCHAR(50)	-	Y	예산 적용 카테고리
amount	DECIMAL(15,2)	-	Y	월간 예산 한도 금액
month	VARCHAR(7)	-	Y	예산 적용 월 (YYYY-MM)

【테이블 관계 (ERD 요약)】

users (1) ■■< (N) transactions users (1) ■■< (N) budgets ■■ category ■■■■
transactions.category ■ ■■■■ ■■

6. 구현 기술 설계

분류	기술/도구	버전	적용 목적
Backend 언어	Java	17+	서버 사이드 비즈니스 로직 구현
Backend 프레임워크	Spring Boot	3.x	REST API 서버, DI, JPA, Security
AI 언어	Python	3.10+	AI 분류 모델 구현 및 FastAPI 서버
AI 프레임워크	FastAPI	최신	AI 서비스 REST API 엔드포인트 제공
AI 모델	scikit-learn / 규칙 기반	-	거래 텍스트 소비 카테고리 분류
Frontend	React + TypeScript	18+	사용자 인터페이스 SPA 구현
데이터베이스	MySQL	8.0+	사용자/거래/예산 데이터 영속 저장
ORM	Spring Data JPA / Hibernate	최신	Java 객체-DB 매핑, 쿼리 추상화
인증	JWT (JSON Web Token)	-	무상태 사용자 인증 및 세션 관리
빌드 도구	Gradle (Backend)	8.x	의존성 관리 및 빌드 자동화
패키지 관리	npm (Frontend)	최신	Frontend 패키지 관리
버전 관리	Git + GitHub	-	소스코드 형상 관리 및 협업
개발 환경	VSCode / IntelliJ IDEA	-	IDE 및 원격 개발 환경 (SSH)
OS / 런타임	Ubuntu Linux	22.04+	서버 및 개발 컨테이너 환경
API 통신	Axios (Frontend)	최신	HTTP 비동기 API 호출 및 인터셉터

【디렉토리 구조】

```

ai-budget-app/
├── backend/ # Spring Boot
│   ├── src/main/java/com/budget/
│   │   ├── controller/ # REST Controller
│   │   ├── service/ #
│   │   └── repository/ # JPA Repository
│   ├── model/ # Entity, DTO
│   ├── build.gradle
│   └── ai/ # Python AI
│       ├── main.py # FastAPI
│       ├── classifier.py #
│       ├── requirements.txt
│       └── frontend/ # React
│           ├── src/
│           ├── pages/ #
│           ├── components/ #
│           ├── api/ # API
│           ├── package.json
│           └── docs/ # README.md

```

7. 요구사항 추적표

아래 표는 요구사항과 설계 요소 간의 추적 관계를 나타낸다.

요구사항 ID	요구사항 설명	관련 설계 요소	구현 모듈
REQ-001	사용자 회원가입 및 로그인	UI-001, IF-003, M-B-005	UserController, UserService, users 테이블
REQ-002	거래 내역 수동 입력	UI-003, IF-003, M-B-001	TransactionController, TransactionForm
REQ-003	AI 기반 소비 자동 분류	IF-001, M-A-001, M-A-002	AI 서비스(classifier.py, preprocessor.py)
REQ-004	카테고리별 예산 설정	UI-004, IF-003, M-B-003	BudgetController, BudgetService, budgets 테이블
REQ-005	월별 소비 분석 조회	UI-005, IF-003, M-B-004	AnalysisController, AnalysisService
REQ-006	거래 목록 조회/필터	UI-003, IF-003, M-B-002	TransactionController, TransactionRepository
REQ-007	거래 수정 및 삭제	UI-003, IF-003	TransactionController (PUT/DELETE)
REQ-008	대시보드 요약 정보 표시	UI-002	Dashboard 페이지, AnalysisService
REQ-009	데이터 영속 저장	DD-01, DD-02, DD-03	MySQL DB, JPA Repository
REQ-010	보안(인증/인가)	4.1 IF-003, M-B-005	Spring Security, JWT 필터

AI 기반 개인 소비 분석 및 예산 관리 앱
소프트웨어 설계서 (SDD-2026-001) | v1.0 | 2026.05.26 | 손재연

<https://github.com/lcanndoit/ai-budget-app>